

Java

The Complete Guide

From platform basics and core syntax through object-oriented design, collections, streams, concurrency, JVM internals and the wider ecosystem, with runnable code examples, comparison tables, interview questions and a condensed summary.

Covers: Java SE fundamentals, OOP, exceptions, generics, collections, lambdas and streams, date/time, I/O, multithreading, virtual threads, memory and GC, JDBC, testing, Spring Boot and modern language features through Java 25.

Contents

1. Introduction to Java	3
2. The Java Platform: JDK, JRE and JVM	4
3. Variables, Data Types and Operators	6
4. Control Flow Statements	8
5. Arrays	9
6. Strings	10
7. Methods	12
8. Classes and Objects	13
9. The Four Pillars of OOP in Depth	14
10. Modifiers, Packages and Imports	16
11. Nested Types, Enums, Records and the Object Class	17
12. Design Principles and Common Patterns	20
13. Exception Handling	22
14. Generics	24
15. The Collections Framework	25
16. Lambdas, Streams and Optional	28
17. Essential Utility APIs	30
18. Input/Output and Files	31
19. Multithreading and Concurrency	33
20. JVM Internals, Memory and Performance	36
21. Networking, Databases, Testing and Logging	38
22. The Java Ecosystem and Version Highlights	40
23. Best Practices and Common Pitfalls	42
24. Frequently Asked Interview Questions	43
25. Learning Roadmap and Practice Projects	46
Condensed Summary	47

1. Introduction to Java

Java is a general-purpose, class-based, object-oriented, statically typed programming language that compiles to bytecode and runs on the Java Virtual Machine (JVM). It was designed by James Gosling and his team at Sun Microsystems, announced in 1995, and first released as Java 1.0 in early 1996. Oracle acquired Sun in 2010. The open-source reference implementation is **OpenJDK**.

1.1 Key Features

Feature	Meaning
Platform independent	Compiled once to bytecode; runs on any OS that has a JVM ("write once, run anywhere").
Object-oriented	Everything except primitives is an object; supports encapsulation, inheritance, polymorphism, abstraction.
Robust	Strong static typing, checked exceptions, automatic garbage collection, no pointer arithmetic.
Secure	Bytecode verifier, class loaders, security manager (deprecated) and a strict runtime model.
Multithreaded	Threads, executors, and a well-defined memory model are built into the language and libraries.
High performance	The JIT compiler optimizes hot code at runtime, often approaching native speed.
Rich ecosystem	Huge standard library plus frameworks (Spring, Hibernate), build tools, and tooling.

1.2 Editions and Distributions

- **Java SE (Standard Edition)**: the core language and libraries. This guide focuses on it.
- **Jakarta EE (formerly Java EE)**: enterprise APIs (servlets, JPA, CDI, REST) for server applications.
- **Java ME**: a legacy edition for small embedded devices. **JavaFX**: a desktop UI toolkit.
- **Distributions of OpenJDK**: Eclipse Temurin, Amazon Corretto, Azul Zulu, Microsoft Build of OpenJDK, Oracle JDK and others. They are functionally very similar; choose one with long-term support.

1.3 Release Model

Since Java 9, a new feature release arrives **every six months** (March and September). Certain releases are designated **LTS (Long-Term Support)** and receive years of updates. The LTS releases to know are **Java 8, 11, 17, 21 and 25**. Most production systems run an LTS release; students should learn on the latest LTS.

1.4 Java Compared with Other Languages

Aspect	Java	C++	Python
Typing	Static, strong	Static	Dynamic
Execution	Bytecode on JVM (JIT)	Native machine code	Interpreted (CPython)
Memory management	Garbage collector	Manual (new/delete, RAII)	Reference counting + GC
Pointers	References only	Full pointers	References only
Multiple inheritance	Interfaces only	Yes (classes)	Yes (classes)
Typical use	Enterprise, Android, big data, backend	Systems, games, performance-critical	Scripting, data science, AI

1.5 Where Java Is Used

- Enterprise backends and REST/microservice systems (banking, insurance, e-commerce).
- Android applications (Java and Kotlin) and large-scale distributed systems.
- Big-data infrastructure: Hadoop, Spark, Kafka, Cassandra, Elasticsearch are JVM-based.
- Trading platforms, telecom, and any domain that values stability, tooling and backward compatibility.

2. The Java Platform: JDK, JRE and JVM

2.1 From Source Code to Execution

```
Hello.java --javac--> Hello.class (bytecode) --JVM--> class loading, verification
                                         -> interpreter + JIT -> native code
```

Component	What it is
JDK	Java Development Kit: compiler (javac), jar , javadoc , jdb , jsell and monitoring tools, plus the runtime.
JRE	Java Runtime Environment: JVM plus standard libraries. Since Java 11 it is no longer distributed separately; install the JDK.
JVM	Specification and implementation that loads classes, verifies bytecode, manages memory, runs code, and performs GC.

2.2 JVM Architecture

- **Class loader subsystem:** loads **.class** files on demand. Built-in loaders: Bootstrap (core JDK classes), Platform (JDK modules) and Application/System (your classpath). Loading follows the parent-delegation model.
- **Linking:** **verification** (bytecode safety checks), **preparation** (allocate static fields with default values), **resolution** (turn symbolic references into direct ones). Then **initialization** runs static initializers.
- **Runtime data areas:** Method Area / Metaspace (class metadata, constants), Heap (all objects, shared), Java Stacks (one per thread; frames hold locals and operand stack), PC Register (per thread) and Native Method Stacks.
- **Execution engine:** the interpreter starts running bytecode immediately; the **JIT compiler** (C1 and C2 in HotSpot) compiles frequently executed methods to optimized native code; the **garbage collector** reclaims memory.
- **JNI / FFM:** interfaces for calling native code (the Foreign Function and Memory API became final in Java 22).

2.3 Installing and Running Your First Program

1. Install a JDK (for example Temurin or Corretto for the LTS version you want).
2. Verify with **java -version** and **javac -version**. Set **JAVA_HOME** and add its **bin** folder to **PATH**.
3. Write, compile and run as below.

```
// Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

```
// Terminal:
// javac Hello.java      -> produces Hello.class
// java Hello            -> prints: Hello, Java!
// java Hello.java       -> Java 11+: compile in memory and run a single source file
```

jsHELL (Java 9+) provides an interactive REPL for trying expressions quickly.

2.4 Program Structure

```
package com.example.app;           // 1. optional package declaration

import java.util.List;             // 2. imports
import java.util.ArrayList;

/** Javadoc comment describing the class. */
public class Demo {                 // 3. class (file must be Demo.java if public)
    private static final int LIMIT = 3; // constant

    public static void main(String[] args) { // 4. entry point
        List<String> items = new ArrayList<>();
        items.add("java");
        System.out.println(items);        // [java]
    }
}
```

- A source file may contain many classes but at most one **public** top-level class, and the file name must match it.
- The entry point signature is **public static void main(String[] args)**. Arguments passed on the command line arrive in **args**.
- Comments: **// line**, **/* block */**, and **/** Javadoc */** (turned into HTML documentation by **javadoc**).

2.5 Naming Conventions

Element	Convention	Example
Class / Interface / Enum / Record	PascalCase (nouns)	BankAccount , Runnable
Method / variable	camelCase (methods are verbs)	calculateTax , totalMarks
Constant (static final)	UPPER_SNAKE_CASE	MAX_RETRIES
Package	all lowercase, reverse domain	com.example.billing
Generic type parameter	single uppercase letter	T , E , K , V

2.6 Reserved Keywords

Java has about 50 reserved keywords that cannot be used as identifiers: **abstract** **assert** **boolean** **break** **byte** **case** **catch** **char** **class** **const** **continue** **default** **do** **double** **else** **enum** **extends** **final** **finally** **float** **for** **goto** **if** **implements** **import** **instanceof** **int** **interface** **long** **native** **new** **package** **private** **protected** **public** **return** **short** **static** **strictfp** **super** **switch** **synchronized** **this** **throw** **throws** **transient** **try** **void** **volatile** **while**. **const** and **goto** are reserved but unused. **true**, **false** and **null** are literals. Words such as **var**, **record**, **sealed**, **yield** are contextual keywords with special meaning only in certain positions.

3. Variables, Data Types and Operators

3.1 Kinds of Variables

Kind	Declared	Lives	Default value
Local	Inside a method/block	Stack (the reference); must be assigned before use	None (compile error if unassigned)
Instance	In a class, no static	Inside each object (heap)	0, false, ' <u>0000</u> ', null
Static (class)	In a class, with static	Once per class (metaspace/heap)	0, false, ' <u>0000</u> ', null
Parameter	Method signature	Stack	Supplied by caller

3.2 Primitive Data Types

Type	Size	Range / Notes	Default
byte	8 bits	-128 to 127	0
short	16 bits	-32,768 to 32,767	0
int	32 bits	about -2.1 billion to 2.1 billion (default integer type)	0
long	64 bits	about -9.2e18 to 9.2e18; literal suffix L	0L
float	32 bits	about 7 decimal digits precision; suffix f	0.0f
double	64 bits	about 15 to 16 digits precision (default decimal type)	0.0
char	16 bits	Unsigned UTF-16 code unit, 0 to 65,535	'\u0000'
boolean	JVM-specific	true or false only	false

```
int million = 1_000_000;    // underscores improve readability
int hex = 0xFF;             // 255
int bin = 0b1010;          // 10
int oct = 017;              // 15 (leading zero = octal!)
long big = 9_000_000_000L;
float f = 3.14f;
double d = 2.5e3;           // 2500.0
char c = 'A';               // also 65 as a number
char u = '\u0041';          // Unicode escape -> 'A'
var name = "Java";          // Java 10+: local type inference (still statically typed)
```

3.3 Type Conversion and Casting

- **Widening (implicit, safe):** **byte** -> **short** -> **int** -> **long** -> **float** -> **double**, and **char** -> **int**.
- **Narrowing (explicit cast, may lose data):** **int** **i** = (**int**) 3.99; gives 3; **byte** **b** = (**byte**) 200; gives -56.
- **Numeric promotion:** in expressions, **byte**, **short** and **char** are promoted to **int** first; if either operand is **long**, **float**, or **double**, the result is that wider type.
- **Compound assignment** (**+=**, **-=**) includes an implicit cast: **byte** **b** = 10; **b** += 5; compiles, but **b** = **b** + 5; does not.

```
int a = 7, b = 2;
```

```

System.out.println(a / b);           // 3      (integer division truncates)
System.out.println(a / (double) b); // 3.5
System.out.println(0.1 + 0.2);       // 0.30000000000000004 (binary floating point)
System.out.println(Integer.MAX_VALUE + 1); // -2147483648 (silent overflow)
System.out.println('a' + 1);         // 98 (char promoted to int)
System.out.println((char) ('a' + 1)); // b

```

Pitfall: Never use `float` or `double` for money. Use `BigDecimal` or store amounts as integer minor units (cents).

3.4 Operators

Category	Operators	Notes
Arithmetic	<code>+ - * / % ++ --</code>	<code>%</code> works on doubles too; <code>+</code> also concatenates Strings
Relational	<code>== != < > <= >=</code>	<code>==</code> on objects compares references
Logical	<code>&& !</code>	<code>&&</code> and <code> </code> short-circuit; <code>&</code> and <code> </code> on booleans do not
Bitwise	<code>& ^ ~</code>	Operate on bits of integral types
Shift	<code><< >> >>></code>	<code>>></code> keeps sign, <code>>>></code> fills with zeros
Assignment	<code>= += -= *= /= %= &= = ^= <<= >>=</code>	Compound forms include an implicit cast
Ternary	<code>cond ? a : b</code>	Expression form of if-else
Type test	<code>instanceof</code>	Supports pattern matching (Java 16+)

Precedence (high to low), simplified: postfix and unary, then `*` `/` `%`, then `+` `-`, shifts, relational, equality, `&`, `^`, `|`, `&&`, `||`, ternary, and finally assignment. When unsure, add parentheses.

```

int x = 5;
int y = x++ + ++x;      // x++ uses 5 then x=6; ++x makes 7 and uses 7 -> y = 12, x = 7
int flags = 0b0101;
System.out.println(flags & 0b0100); // 4
System.out.println(flags << 2);     // 20
System.out.println(-16 >> 2);       // -4 (sign preserved)
System.out.println(-16 >>> 28);      // 15 (zero fill)
String grade = (x > 6) ? "high" : "low";

```

3.5 Wrapper Classes and Autoboxing

Each primitive has a wrapper class (`Integer`, `Long`, `Double`, `Boolean`, `Character`, `Byte`, `Short`, `Float`). Collections and generics work only with these. Java automatically converts between primitive and wrapper (**autoboxing** and **unboxing**). Wrappers are immutable and provide helpers such as `Integer.parseInt`, `Integer.valueOf`, `Integer.compare`, `Double.isNaN`, `Character.isDigit`.

```

Integer a = 127, b = 127;
System.out.println(a == b); // true (small values are cached: -128..127)
Integer c = 1000, d = 1000;
System.out.println(c == d); // false (different objects)
System.out.println(c.equals(d)); // true
Integer n = null;
int m = n; // NullPointerException on unboxing

```

3.6 Console Input and Formatted Output

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Name: ");
String name = sc.nextLine();
System.out.print("Age: ");
int age = sc.nextInt();
System.out.printf("Hello %s, next year you will be %d\n", name, age + 1);
String row = String.format("%08.3f | %-8s| %5d | %x | %,d", 3.14159, "left", 42, 255, 1234567);
System.out.println(row);
// 0003.142 | left      |    42 | ff | 1,234,567
sc.close();
```

Specifier	Meaning	Specifier	Meaning
%d	integer	%s	string / any object
%f %.2f	floating point	%c	character
%x	hexadecimal	%b	boolean
%n	platform line separator	%, d	grouping separators

4. Control Flow Statements

4.1 Decision Making

```
int marks = 78;
if (marks >= 90) {
    System.out.println("A");
} else if (marks >= 75) {
    System.out.println("B");
} else {
    System.out.println("C");
}
```

switch (classic and modern)

```
// Classic switch: needs break, otherwise falls through
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 6:
    case 7: System.out.println("Weekend"); break;
    default: System.out.println("Weekday");
}

// Switch expression (Java 14+): no fall-through, returns a value, must be exhaustive
String type = switch (day) {
    case 6, 7 -> "Weekend";
    case 1, 2, 3, 4, 5 -> "Weekday";
    default -> throw new IllegalArgumentException("Bad day: " + day);
};

// Block form uses yield
int len = switch (name) {
    case "Java" -> 4;
    default -> { int n = name.length(); yield n; }
};

// Pattern matching for switch (Java 21)
```

```

static String describe(Object o) {
    return switch (o) {
        case Integer i when i > 10 -> "big int " + i;
        case Integer i             -> "int " + i;
        case String s              -> "string of length " + s.length();
        case null                  -> "null!";
        default                    -> "other";
    };
}

```

`switch` supports `byte`, `short`, `char`, `int`, their wrappers, `String`, and `enum` types (and any object with pattern matching in Java 21). It does not support `long`, `float`, `double` or `boolean`.

4.2 Loops

```

for (int i = 0; i < 5; i++) System.out.print(i + " ");          // 0 1 2 3 4

int n = 3;
while (n > 0) { System.out.print(n-- + " "); }                 // 3 2 1

int k = 10;
do { System.out.println("runs at least once: " + k); } while (k < 5);

int[] nums = {4, 5, 6};
for (int v : nums) System.out.print(v + " ");                  // enhanced for

outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 2) continue outer;          // jump to the next i
        if (i == 2) break outer;             // leave both loops
        System.out.println(i + "," + j);
    }
}

```

- `break` exits the nearest loop or switch; `continue` skips to the next iteration; labels target an outer loop.
- The enhanced `for-each` works with arrays and any `Iterable`. You cannot modify a collection while iterating with it (use an `Iterator` or `removeIf`).
- An infinite loop is written `while (true)` or `for (;;)` and is normally exited with `break` or `return`.

5. Arrays

An array is a fixed-size, zero-indexed, ordered container of elements of a single type. Arrays are **objects** stored on the heap, have a `length` field (not a method), and their elements receive default values.

```

int[] a = new int[5];          // [0, 0, 0, 0, 0]
int[] b = {3, 1, 2};
String[] names = new String[3]; // [null, null, null]
int[][] grid = new int[3][4];   // 3 rows, 4 columns
int[][] tri = new int[3][];     // jagged: rows created individually
for (int i = 0; i < tri.length; i++) tri[i] = new int[i + 1];
int[][] m = { {1, 2}, {3, 4} };

System.out.println(b.length);  // 3
int x = b[3];                  // ArrayIndexOutOfBoundsException

```

5.1 The Arrays Utility Class

```
import java.util.Arrays;

int[] a = {5, 2, 9, 1};
Arrays.sort(a); // [1, 2, 5, 9] (dual-pivot quicksort)
int idx = Arrays.binarySearch(a, 5); // 2 (array must be sorted)
int[] copy = Arrays.copyOf(a, 6); // [1, 2, 5, 9, 0, 0]
int[] part = Arrays.copyOfRange(a, 1, 3); // [2, 5]
Arrays.fill(a, 7); // [7, 7, 7, 7]
System.out.println(Arrays.toString(a)); // [7, 7, 7, 7]
System.out.println(Arrays.deepToString(new int[][]{{1}, {2, 3}})); // [[1], [2, 3]]
System.out.println(Arrays.equals(new int[]{1}, new int[]{1})); // true
List<Integer> list = Arrays.asList(1, 2, 3); // fixed-size list backed by the array
int total = Arrays.stream(new int[]{1, 2, 3}).sum(); // 6
```

Pitfall: `a == b` on arrays compares references and `a.toString()` prints something like `[I@1b6d3586`. Use `Arrays.equals` and `Arrays.toString`.

- Arrays are **covariant**: `Object[] o = new String[1]; o[0] = 1;` compiles but throws `ArrayStoreException` at runtime. Generics are invariant and avoid this.
- Arrays cannot grow. Use `ArrayList` when the size changes.
- Arrays of generics cannot be created directly (`new List<String>[3]` is illegal).

6. Strings

6.1 Immutability and the String Pool

A **String** is an immutable sequence of UTF-16 characters. String literals are stored in the **string pool** so identical literals share one object. `new String("x")` always creates a separate heap object; `intern()` returns the pooled version.

```
String s1 = "java";
String s2 = "java";
String s3 = new String("java");
System.out.println(s1 == s2); // true
System.out.println(s1 == s3); // false
System.out.println(s1.equals(s3)); // true
System.out.println(s1 == s3.intern()); // true
String t = s1.concat(" rocks"); // new object; s1 is unchanged
```

Immutability gives thread safety, safe hash-key use (the hash code is cached), security and pooling.

6.2 Commonly Used Methods

Method	Description	Example result
<code>length()</code> <code>charAt(i)</code>	Size and character at index	<code>"java".charAt(1)</code> is 'a'
<code>substring(a, b)</code>	Slice from a (inclusive) to b (exclusive)	<code>"hello".substring(1, 3)</code> is "el"
<code>indexOf</code> <code>lastIndexOf</code>	Position of text, -1 if absent	<code>"banana".indexOf("an")</code> is 1
<code>contains</code> <code>startsWith</code> <code>endsWith</code>	Boolean tests	<code>"java".contains("av")</code> is true

Method	Description	Example result
<code>equals equalsIgnoreCase</code>	Content comparison	<code>"A".equalsIgnoreCase("a")</code> is true
<code>compareTo</code>	Lexicographic order (negative, 0, positive)	<code>"a".compareTo("b")</code> is -1
<code>toUpperCase toLowerCase</code>	Case conversion (returns new String)	
<code>trim strip</code>	Remove whitespace (<code>strip</code> is Unicode-aware, Java 11)	
<code>isEmpty isBlank</code>	Length 0 / only whitespace (Java 11)	
<code>replace replaceAll</code>	Replace text / regex	
<code>split(regex)</code>	Break into <code>String[]</code>	<code>"a,b".split(",")</code> gives 2 parts
<code>join repeat lines</code>	Combine, repeat, split into lines (Java 8/11)	<code>"ab".repeat(2)</code> is "abab"
<code>toCharArray valueOf</code>	Convert to and from other types	
<code>format formatted</code>	printf-style formatting	<code>"%05d".formatted(42)</code> is "00042"

6.3 StringBuilder and StringBuffer

	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Speed for repeated changes	Slow (new object each time)	Fastest	Slower than StringBuilder

```

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) sb.append(i).append(',');
sb.setLength(sb.length() - 1);           // remove last comma
sb.insert(0, "[").append("]");
System.out.println(sb);                  // [0,1,2,3,4]
System.out.println(sb.reverse());        // ]4,3,2,1,0[

```

Tip: Concatenating with `+` inside a loop creates many temporary objects. Use `StringBuilder` when building strings in loops.

6.4 Text Blocks (Java 15+) and Regular Expressions

```

String json = """
    {
      "name": "Asha",
      "score": 95
    }
    """;                                     // multi-line string, indentation is stripped

import java.util.regex.*;

Pattern p = Pattern.compile("(\\w+)@(\\w+)\\.com");
Matcher m = p.matcher("mail asha@example.com now");
if (m.find()) {
    System.out.println(m.group(1));          // asha
}

```

```

        System.out.println(m.group(2));          // example
    }
    System.out.println("a1b22c".replaceAll("[0-9]+", "#"));    // a#b#c
    System.out.println("2026-09-19".matches("\\d{4}-\\d{2}-\\d{2}")); // true

```

7. Methods

A method is a named block of code with an optional parameter list and return type. It declares behavior of a class and promotes reuse.

```

[modifiers] returnType name(paramType param, ...) [throws ExceptionType] {
    // body
    return value;
}

static int add(int a, int b) { return a + b; }
static void greet(String name) { System.out.println("Hi " + name); }

```

7.1 Pass-by-Value

Java is **always pass-by-value**. For primitives the value is copied. For objects the **reference** is copied, so the method can modify the object's state but cannot make the caller's variable point to a different object.

```

static void change(int n, StringBuilder sb, int[] arr) {
    n = 99;                      // caller's int unaffected
    sb.append("!");              // object modified: caller sees it
    arr[0] = 42;                 // array modified: caller sees it
    sb = new StringBuilder("new"); // only the local copy of the reference changes
}

int n = 1; StringBuilder sb = new StringBuilder("hi"); int[] arr = {0};
change(n, sb, arr);
System.out.println(n + " " + sb + " " + arr[0]);    // 1 hi! 42

```

7.2 Overloading and Varargs

```

static int sum(int a, int b)      { return a + b; }
static double sum(double a, double b) { return a + b; }
static int sum(int... nums) {      // varargs: zero or more ints, treated as int[]
    int t = 0;
    for (int n : nums) t += n;
    return t;
}

sum(1, 2);           // picks (int,int)
sum(1.5, 2);         // picks (double,double)
sum(1, 2, 3, 4);     // picks varargs

```

Overload resolution order: exact match, then widening, then boxing, then varargs. Only one varargs parameter is allowed and it must be last.

7.3 Recursion

```

static long factorial(int n) {
    if (n <= 1) return 1;          // base case is mandatory
    return n * factorial(n - 1);
}

static long fib(int n, long[] memo) { // memoization avoids exponential time
    if (n < 2) return n;
}

```

```

        if (memo[n] != 0) return memo[n];
        return memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
    }
    System.out.println(factorial(10));           // 3628800
    System.out.println(fib(50, new long[51]));   // 12586269025

```

Warning: Java does not optimize tail calls. Very deep recursion throws `StackOverflowError`; prefer iteration or an explicit stack for large depths.

8. Classes and Objects

A **class** defines state (fields) and behavior (methods); an **object** is a runtime instance. Objects are created with **new**, live on the heap, and are reclaimed by the garbage collector when unreachable.

```

public class BankAccount {
    private static int totalAccounts = 0;    // shared by all objects
    private final String id;                 // set once
    private double balance;                  // instance state

    static { System.out.println("class loaded"); }    // static initializer: once per class
    { totalAccounts++; }    // instance initializer: every object

    public BankAccount(String id) { this(id, 0.0); }    // constructor chaining
    public BankAccount(String id, double opening) {
        this.id = id;
        this.balance = opening;
    }

    public void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("Amount must be positive");
        balance += amt;
    }
    public double getBalance() { return balance; }
    public static int getTotalAccounts() { return totalAccounts; }
}

BankAccount acc = new BankAccount("A-101", 500);
acc.deposit(250);
System.out.println(acc.getBalance());           // 750.0
System.out.println(BankAccount.getTotalAccounts()); // 1

```

8.1 Order of Initialization

1. When a class is first used: parent static blocks/fields, then child static blocks/fields (once).
2. For each **new**: parent instance initializers and fields, parent constructor body; then child instance initializers and fields, child constructor body.
3. Fields get default values first, then explicit initializers run in source order.

8.2 Constructors

- Same name as the class, no return type; may be overloaded and may be **private**, **protected**, or **public**.
- If none is written, the compiler adds a default no-argument constructor.
- **this(...)** calls another constructor of the same class; **super(...)** calls a parent constructor. Traditionally either must be the first statement; newer releases (flexible constructor bodies, finalized in Java 25) allow limited statements before it that do not use **this**.
- Java has no destructors. Use **try-with-resources** and **AutoCloseable** for deterministic cleanup.

- A **copy constructor** is just a convention: `Point(Point other) { this(other.x, other.y); }`.

8.3 The `this` Keyword and Method Chaining

```
class Counter {
    private int n;
    Counter inc() { n++; return this; }    // returning this enables chaining
    int get() { return n; }
}
System.out.println(new Counter().inc().inc().inc().get());    // 3
```

8.4 Static vs Instance Members

	Static	Instance
Belongs to	The class (one copy)	Each object
Accessed via	<code>ClassName.member</code>	<code>objectRef.member</code>
Can use <code>this</code>	No	Yes
Can access	Only static members directly	Static and instance members
Typical use	Utilities, constants, counters, factories	Object state and behavior

9. The Four Pillars of OOP in Depth

9.1 Encapsulation

Bundle data with the methods that operate on it and hide internal state. Fields are `private`; access is granted through methods that can validate, log, or compute. Benefits: invariants can be enforced, internals can change without breaking callers, and coupling is reduced.

```
public class Temperature {
    private double celsius;
    public double getCelsius() { return celsius; }
    public void setCelsius(double c) {
        if (c < -273.15) throw new IllegalArgumentException("Below absolute zero");
        this.celsius = c;
    }
    public double getFahrenheit() { return celsius * 9 / 5 + 32; }    // computed property
}
```

9.2 Inheritance

A subclass acquires the non-private members of its superclass with `extends`, modelling an **IS-A** relationship. Java supports single inheritance of classes; every class extends `Object`.

```
class Vehicle {
    protected int speed;
    Vehicle(int speed) { this.speed = speed; }
    void move() { System.out.println("Moving at " + speed); }
}

class Car extends Vehicle {
    private final int doors;
    Car(int speed, int doors) {
        super(speed);    // parent constructor first
        this.doors = doors;
    }
}
```

```

@Override
void move() {
    super.move();                // reuse parent behavior
    System.out.println("Car with " + doors + " doors");
}
}

```

- Private members exist in the object but are not accessible in the subclass. Constructors are not inherited.
- **final** classes (e.g. **String**) cannot be extended; **final** methods cannot be overridden.
- Prefer **composition over inheritance** when the relationship is HAS-A or when you only want to reuse code: inheritance couples the subclass tightly to the parent's implementation.

9.3 Polymorphism

Compile-time (static) polymorphism: method overloading and operator-like overloading of **+** for Strings.

Runtime (dynamic) polymorphism: a superclass reference can hold any subclass object, and the overridden method of the actual object runs.

```

abstract class Shape { abstract double area(); }
class Circle extends Shape {
    final double r; Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }
}
class Rect extends Shape {
    final double w, h; Rect(double w, double h) { this.w = w; this.h = h; }
    double area() { return w * h; }
}

```

```

Shape[] shapes = { new Circle(1), new Rect(2, 3) };
for (Shape s : shapes) System.out.printf("%.2f\n", s.area());    // 3.14, 6.00

```

```

Object o = shapes[0];
if (o instanceof Circle c) System.out.println("radius " + c.r);    // pattern matching
Circle back = (Circle) o;      // downcast; ClassCastException if wrong

```

Rule for overriding	Detail
Signature	Same name and parameter types
Return type	Same or a subtype (covariant return)
Access	Same or more visible; cannot be reduced
Checked exceptions	Same, narrower, or none; never broader
Not overridable	static (hidden), private , final methods, constructors
Fields	Not polymorphic; fields are hidden and resolved by reference type

9.4 Abstraction

Expose what an object does and hide how it does it. Java provides **abstract classes** and **interfaces**.

```

abstract class Report {
    public final void generate() {                // template method: fixed algorithm
        header();
        body();                                  // varies by subclass
        footer();
    }
    protected void header() { System.out.println("== Report =="); }
    protected abstract void body();
    protected void footer() { System.out.println("== End =="); }
}

```

```
}
```

9.5 Interfaces

```
interface Payable {
    double MAX_LIMIT = 100_000;           // implicitly public static final
    double amount();                       // implicitly public abstract

    default String receipt() {             // Java 8: default method
        return "Paid: " + amount();
    }
    static Payable of(double a) { return () -> a; } // factory; Payable is functional
    private String secret() { return "internal"; }   // Java 9: private helper
}

class Invoice implements Payable, Comparable<Invoice> {
    private final double total;
    Invoice(double total) { this.total = total; }
    public double amount() { return total; }
    public int compareTo(Invoice o) { return Double.compare(total, o.total); }
}
```

Feature	Abstract class	Interface
Instance state (fields)	Yes	No (constants only)
Constructors	Yes	No
Multiple inheritance	No (extend one)	Yes (implement many, extend many)
Method kinds	Abstract, concrete, static, final	Abstract, default , static , private
Default member access	Package-private	public
Best for	Shared code/state among closely related types	Capabilities and contracts across unrelated types

- A **marker interface** has no methods (**Serializable**, **Cloneable**) and tags a class. Annotations are the modern alternative.
- A **functional interface** has exactly one abstract method and can be implemented by a lambda.
- If two interfaces provide the same default method, the implementing class must override it and can choose with **InterfaceName.super.method()**.

10. Modifiers, Packages and Imports

10.1 Access Modifiers

Modifier	Same class	Same package	Subclass (other package)	Anywhere
private	Yes	No	No	No
(default)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Top-level classes may only be **public** or package-private. Prefer the **most restrictive** access that works.

10.2 Non-Access Modifiers

Modifier	Applies to	Effect
<code>static</code>	Fields, methods, nested classes, blocks	Belongs to the class, not an instance
<code>final</code>	Variables, methods, classes	Constant / not overridable / not extendable
<code>abstract</code>	Classes, methods	Incomplete; must be extended/implemented
<code>synchronized</code>	Methods, blocks	Acquires an intrinsic lock for mutual exclusion
<code>volatile</code>	Fields	Guarantees visibility and ordering across threads (not atomicity)
<code>transient</code>	Fields	Skipped during default serialization
<code>native</code>	Methods	Implemented in native code via JNI
<code>default</code>	Interface methods	Provides an implementation in an interface

Pitfall: `final` on a reference means the reference cannot change, not that the object is immutable. A `final List` can still be modified.

10.3 Packages and Imports

```
package com.shop.billing;           // file must live in com/shop/billing/

import java.util.List;              // single-type import
import java.util.*;                 // on-demand import (no subpackages)
import static java.lang.Math.*;     // static import: sqrt(4) instead of Math.sqrt(4)

// Compile from source root: javac -d out com/shop/billing/*.java
// Run:                       java -cp out com.shop.billing.Main
```

- `java.lang` is imported automatically (String, Math, System, Object, wrapper classes).
- Packages prevent name clashes and group related types. Use reverse domain names (`com.company.project`).
- Java 9 **modules (JPMS)** add a layer above packages: a `module-info.java` declares what a module **requires** and **exports**.

11. Nested Types, Enums, Records and the Object Class

11.1 Nested and Inner Classes

Type	Declaration	Key points
Static nested	<code>static class N</code> inside a class	No outer instance; behaves like a top-level class in a namespace
Inner (member)	<code>class I</code> inside a class	Holds an implicit reference to the outer instance and can use its members
Local	Class declared inside a method	Can capture effectively final local variables
Anonymous	<code>new Type() { ... }</code>	One-off unnamed subclass/implementation; lambdas replace most uses

```

class Outer {
    private int x = 10;
    class Inner { int twice() { return x * 2; } }           // needs an Outer instance
    static class Nested { int one() { return 1; } }        // does not
}
Outer o = new Outer();
Outer.Inner in = o.new Inner();
Outer.Nested ne = new Outer.Nested();

Runnable r = new Runnable() {                           // anonymous class
    @Override public void run() { System.out.println("run"); }
};

```

11.2 Enums

An enum is a fixed set of named constants, implemented as a class with a finite number of instances. Enums can have fields, constructors, methods, implement interfaces, and define constant-specific bodies. They are thread-safe singletons and work well in **switch**.

```

enum Planet {
    MERCURY(3.303e+23, 2.4397e6),
    EARTH(5.976e+24, 6.37814e6);

    private final double mass, radius;
    Planet(double mass, double radius) { this.mass = mass; this.radius = radius; }
    double surfaceGravity() { return 6.67300E-11 * mass / (radius * radius); }
}

enum Op {
    ADD("+") { int apply(int a, int b) { return a + b; } },
    MUL("*") { int apply(int a, int b) { return a * b; } },
    final String symbol;
    Op(String s) { symbol = s; }
    abstract int apply(int a, int b);
}

for (Planet p : Planet.values()) System.out.println(p + " " + p.ordinal());
Planet e = Planet.valueOf("EARTH");
EnumMap<Planet, String> notes = new EnumMap<>(Planet.class); // fast, ordered by ordinal

```

11.3 Records (Java 16+)

A record is a concise **immutable data carrier**. The compiler generates the private final fields, canonical constructor, accessors (**name()**, not **getName()**), **equals**, **hashCode** and **toString**. Records are implicitly **final** and cannot extend another class, but can implement interfaces.

```

record Point(int x, int y) {
    Point {                                               // compact constructor: validation
        if (x < 0 || y < 0) throw new IllegalArgumentException("negative");
    }
    double distance() { return Math.sqrt(x * x + y * y); }
    static Point origin() { return new Point(0, 0); }
}

Point p = new Point(3, 4);
System.out.println(p);                                // Point[x=3, y=4]
System.out.println(p.distance());                      // 5.0
System.out.println(p.equals(new Point(3, 4)));        // true

// Record patterns (Java 21)
Object o = p;
if (o instanceof Point(int a, int b)) System.out.println(a + b); // 7

```

11.4 Sealed Classes (Java 17+)

```
sealed interface Shape permits Circle, Square, Triangle {}
record Circle(double r) implements Shape {}
record Square(double s) implements Shape {}
record Triangle(double b, double h) implements Shape {}

static double area(Shape s) {
    return switch (s) {
        // exhaustive: no default needed
        case Circle c   -> Math.PI * c.r() * c.r();
        case Square q    -> q.s() * q.s();
        case Triangle t -> 0.5 * t.b() * t.h();
    };
}
```

sealed restricts which classes may extend or implement a type. Permitted subclasses must be **final**, **sealed**, or **non-sealed**. This enables exhaustive **switch** over a closed hierarchy.

11.5 The Object Class

Method	Purpose and guidance
<code>equals(Object)</code>	Logical equality. Default compares references. Override together with <code>hashCode</code> .
<code>hashCode()</code>	Hash for hash-based collections. Equal objects must return equal hash codes.
<code>toString()</code>	Readable representation. Default is <code>ClassName@hex</code> . Override it for debugging and logging.
<code>getClass()</code>	Runtime class object; used by reflection.
<code>clone()</code>	Shallow copy for <code>Cloneable</code> classes; error-prone. Prefer copy constructors or factories.
<code>wait/notify/notifyAll</code>	Low-level thread coordination inside <code>synchronized</code> blocks.
<code>finalize()</code>	Deprecated for removal. Use <code>try-with-resources</code> or <code>Cleaner</code> instead.

```
import java.util.Objects;

class Student {
    private final int id;
    private final String name;
    Student(int id, String name) { this.id = id; this.name = name; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student s)) return false;
        return id == s.id && Objects.equals(name, s.name);
    }
    @Override public int hashCode() { return Objects.hash(id, name); }
    @Override public String toString() { return "Student{id=" + id + ", name=" + name + "}"; }
}
```

11.6 Immutable Classes and Copying

- **Immutable class recipe:** **final** class, **private final** fields, no setters, validate in the constructor, make defensive copies of mutable inputs and outputs. Records do most of this for you.
- **Shallow copy** copies field values (references are shared). **Deep copy** recursively copies referenced objects. Implement copy constructors or use serialization/mapping libraries for deep copies.

```
final class Schedule {
```

```

private final List<String> slots;
Schedule(List<String> slots) { this.slots = List.copyOf(slots); } // defensive copy
List<String> slots() { return slots; } // safe: unmodifiable
}

```

12. Design Principles and Common Patterns

12.1 SOLID Principles

Principle	Meaning	Practical hint
Single Responsibility	A class has one reason to change	Separate persistence, validation and presentation
Open/Closed	Open for extension, closed for modification	Add new behavior via new subclasses/strategies
Liskov Substitution	Subtypes must be usable wherever the base type is	Do not strengthen preconditions in overrides
Interface Segregation	Many small interfaces beat one large one	Clients should not depend on unused methods
Dependency Inversion	Depend on abstractions, not concretions	Inject interfaces (constructor injection)

Other guidelines: **DRY** (do not repeat yourself), **KISS** (keep it simple), **YAGNI** (you aren't gonna need it), **favor composition over inheritance**, and **program to an interface**.

12.2 Singleton

```

public enum AppConfig { // enum singleton: thread-safe and serialization-safe
    INSTANCE;
    private final Map<String, String> props = new java.util.concurrent.ConcurrentHashMap<>();
    public String get(String k) { return props.get(k); }
    public void set(String k, String v) { props.put(k, v); }
}
AppConfig.INSTANCE.set("env", "prod");

// Lazy holder idiom (class-based)
public class Registry {
    private Registry() {}
    private static class Holder { static final Registry INSTANCE = new Registry(); }
    public static Registry getInstance() { return Holder.INSTANCE; }
}

```

12.3 Factory

```

interface Notifier { void send(String msg); }
class EmailNotifier implements Notifier {
    public void send(String m) { System.out.println("Email: " + m); }
}
class SmsNotifier implements Notifier {
    public void send(String m) { System.out.println("SMS: " + m); }
}

class NotifierFactory {
    static Notifier create(String type) {
        return switch (type) {
            case "email" -> new EmailNotifier();
            case "sms" -> new SmsNotifier();
        };
    }
}

```

```

        default -> throw new IllegalArgumentException("Unknown type: " + type);
    };
}
}
NotifierFactory.create("sms").send("Hello");

```

12.4 Builder

```

class Pizza {
    private final String size; private final boolean cheese; private final List<String> toppings;
    private Pizza(Builder b) {
        size = b.size; cheese = b.cheese; toppings = List.copyOf(b.toppings);
    }

    static class Builder {
        private String size = "M"; private boolean cheese;
        private final List<String> toppings = new ArrayList<>();
        Builder size(String s) { this.size = s; return this; }
        Builder cheese(boolean c) { this.cheese = c; return this; }
        Builder topping(String t) { toppings.add(t); return this; }
        Pizza build() { return new Pizza(this); }
    }
}

Pizza p = new Pizza.Builder().size("L").cheese(true).topping("olives").topping("corn").build();

```

12.5 Strategy

```

interface Discount { double apply(double price); }

class Checkout {
    private final Discount discount;
    Checkout(Discount d) { this.discount = d; }
    double total(double price) { return discount.apply(price); }
}

Checkout festive = new Checkout(price -> price * 0.80); // lambda as a strategy
Checkout none = new Checkout(price -> price);
System.out.println(festive.total(100)); // 80.0

```

12.6 Observer

```

interface Listener<T> { void onEvent(T event); }

class EventBus<T> {
    private final List<Listener<T>> listeners = new java.util.concurrent.CopyOnWriteArrayList<>();
    void subscribe(Listener<T> l) { listeners.add(l); }
    void publish(T event) { listeners.forEach(l -> l.onEvent(event)); }
}

EventBus<String> bus = new EventBus<>();
bus.subscribe(e -> System.out.println("Got " + e));
bus.publish("order-created");

```

13. Exception Handling

An exception is an object that represents an abnormal event during execution. Java separates normal logic from error handling using `try`, `catch`, `finally`, `throw` and `throws`.

13.1 The Hierarchy

```
Throwable
|-- Error                      (JVM problems; do not catch)  OutOfMemoryError, StackOverflowError
`-- Exception
    |-- IOException, SQLException, InterruptedException ...  CHECKED (compiler forces handling)
    `-- RuntimeException                                           UNCHECKED
        |-- NullPointerException, IllegalArgumentException, IllegalStateException
        |-- ArrayIndexOutOfBoundsException, ClassCastException, ArithmeticException
        |-- NumberFormatException, UnsupportedOperationException
        `-- ConcurrentModificationException
```

	Checked	Unchecked
Base	<code>Exception</code> (not <code>RuntimeException</code>)	<code>RuntimeException</code> and <code>Error</code>
Compiler check	Must catch or declare with <code>throws</code>	No requirement
Meaning	Recoverable, external conditions (I/O, DB, network)	Programming errors (bugs) or unrecoverable failures

13.2 try, catch, finally

```
try {
    int r = Integer.parseInt(text) / divisor;
} catch (NumberFormatException | ArithmeticException e) {    // multi-catch (unrelated types)
    System.out.println("Bad input: " + e.getMessage());
} catch (Exception e) {                                     // broader types go last
    throw new IllegalStateException("Unexpected", e);        // wrap and keep the cause
} finally {
    System.out.println("always runs (cleanup)");
}
```

- `catch` blocks are checked top to bottom; a subclass handler placed after its superclass is a compile error.
- `finally` runs whether or not an exception occurs, even on `return`. It does not run if the JVM exits (`System.exit`) or crashes.
- Never `return` from `finally`: it silently swallows exceptions and overrides the try block's return value.

```
static int tricky() {
    try { return 1; }
    finally { System.out.println("cleanup"); }    // prints, then 1 is returned
}
```

13.3 try-with-resources

Any object implementing `AutoCloseable` can be declared in the `try (...)` header and is closed automatically in reverse order, even when an exception occurs. Exceptions thrown by `close()` are attached as **suppressed** exceptions.

```
try (var in = new java.io.BufferedReader(new java.io.FileReader("in.txt"));
     var out = new java.io.PrintWriter("out.txt")) {
    String line;
    while ((line = in.readLine()) != null) out.println(line.toUpperCase());
}
```

```

} // out.close() then in.close() are called automatically

class Connection implements AutoCloseable {
    @Override public void close() { System.out.println("closed"); }
}

```

13.4 Throwing and Custom Exceptions

```

class InsufficientFundsException extends Exception { // checked
    private final double shortfall;
    InsufficientFundsException(String msg, double shortfall) {
        super(msg);
        this.shortfall = shortfall;
    }
    double getShortfall() { return shortfall; }
}

void withdraw(double amt) throws InsufficientFundsException {
    if (amt > balance) throw new InsufficientFundsException("Insufficient funds", amt - balance);
    balance -= amt;
}

```

13.5 Common Exceptions

Exception	Typical cause
<code>NullPointerException</code>	Using a null reference (call, field access, unboxing, <code>switch</code> on null)
<code>ArrayIndexOutOfBoundsException</code> / <code>IndexOutOfBoundsException</code>	Index outside the valid range
<code>ClassCastException</code>	Invalid downcast
<code>NumberFormatException</code>	Parsing a non-numeric string
<code>ArithmeticException</code>	Integer division by zero
<code>IllegalArgumentException</code> / <code>IllegalStateException</code>	Bad argument / object in the wrong state
<code>ConcurrentModificationException</code>	Modifying a collection while iterating it
<code>StackOverflowError</code>	Runaway recursion
<code>OutOfMemoryError</code>	Heap (or metaspace, native memory) exhausted
<code>ClassNotFoundException</code> / <code>NoClassDefFoundError</code>	Class missing at load time

13.6 Best Practices

- Catch the most specific exception you can handle. Do not catch `Throwable/Exception` broadly unless at a boundary (e.g., a request handler).
- Never swallow exceptions with an empty `catch`. Log with context or rethrow.
- Use unchecked exceptions for programming errors; checked ones only when callers can realistically recover.
- Preserve the root cause when wrapping (`new RuntimeException("msg", cause)`).
- If you catch `InterruptedException`, restore the flag: `Thread.currentThread().interrupt();`.
- Validate arguments early with `Objects.requireNonNull` and clear messages. Do not use exceptions for normal control flow.

- Use `assert` only for internal sanity checks (disabled by default; enable with `-ea`).

14. Generics

Generics parameterize classes, interfaces and methods by type, giving compile-time type safety and removing explicit casts.

```
// Without generics (Java 1.4 style): needs casts, unsafe
List raw = new ArrayList();
raw.add("a"); raw.add(1);
String s = (String) raw.get(1); // ClassCastException at runtime

// With generics: errors move to compile time
List<String> names = new ArrayList<>(); // diamond operator infers <String>
names.add("Asha");
// names.add(5); // compile error
String first = names.get(0); // no cast
```

14.1 Generic Classes, Interfaces and Methods

```
class Pair<A, B> {
    private final A first; private final B second;
    Pair(A first, B second) { this.first = first; this.second = second; }
    A first() { return first; }
    B second() { return second; }
    static <X, Y> Pair<X, Y> of(X x, Y y) { return new Pair<>(x, y); } // generic static method
}
Pair<String, Integer> p = Pair.of("age", 21);

interface Repository<T, ID> {
    T findById(ID id);
    List<T> findAll();
    void save(T entity);
}

static <T extends Comparable<T>> T max(List<T> items) { // bounded type parameter
    T best = items.get(0);
    for (T t : items) if (t.compareTo(best) > 0) best = t;
    return best;
}
```

14.2 Wildcards and PECS

Syntax	Meaning	Use
<code>List<?></code>	List of unknown type	Read as Object; cannot add (except null)
<code>List<? extends Number></code>	List of Number or a subtype (upper bound)	Producer: safe to read Numbers, cannot add
<code>List<? super Integer></code>	List of Integer or a supertype (lower bound)	Consumer: safe to add Integers, reads return Object

```
static double sum(Collection<? extends Number> src) { // producer extends
    double t = 0; for (Number n : src) t += n.doubleValue(); return t;
}
static void fill(List<? super Integer> dst) { // consumer super
    for (int i = 0; i < 3; i++) dst.add(i);
}
// PECS: Producer Extends, Consumer Super. Collections.copy(dest, src) follows this rule.
```

Note: `List<Integer>` is **not** a subtype of `List<Number>`. Generics are invariant; use wildcards for flexibility.

14.3 Type Erasure and Its Consequences

- The compiler removes type arguments after checking and inserts casts. At runtime `List<String>` and `List<Integer>` are both `List`.
- You cannot write `new T()`, `new T[n]`, `T.class`, `instanceof List<String>`, or static fields of type `T`.
- Primitives are not allowed as type arguments: use `List<Integer>`.
- Two overloads that differ only by type argument clash after erasure: `void f(List<String>)` vs `void f(List<Integer>)`.
- Mixing raw and generic types can cause **heap pollution** and unchecked warnings. Avoid raw types.
- `@SafeVarargs` and `@SuppressWarnings("unchecked")` should be used sparingly and only when you have verified safety.

15. The Collections Framework

15.1 Hierarchy

```
Iterable
  |-- Collection
    |-- List      : ArrayList, LinkedList, Vector, Stack, CopyOnWriteArrayList
    |-- Set       : HashSet, LinkedHashSet, TreeSet (SortedSet/NavigableSet), EnumSet
    |-- Queue     : PriorityQueue, ArrayDeque (Deque), LinkedList, BlockingQueue types

Map (separate hierarchy): HashMap, LinkedHashMap, TreeMap (SortedMap/NavigableMap),
                          Hashtable, ConcurrentHashMap, EnumMap, WeakHashMap, IdentityHashMap
```

15.2 Implementations at a Glance

Class	Backing structure	Order	Null	Typical cost	Notes
<code>ArrayList</code>	Resizable array	Insertion	Yes	get O(1), add end O(1)*, insert O(n)	Default List choice
<code>LinkedList</code>	Doubly linked list	Insertion	Yes	ends O(1), get O(n)	Also a Deque; rarely best
<code>HashSet</code>	HashMap	None	One null	add/contains O(1)*	Needs <code>hashCode/equals</code>
<code>LinkedHashSet</code>	Hash + linked list	Insertion	One null	O(1)*	Predictable iteration
<code>TreeSet</code>	Red-black tree	Sorted	No	O(log n)	<code>first</code> , <code>floor</code> , <code>ceiling</code>
<code>HashMap</code>	Array of buckets	None	1 null key	get/put O(1)*	Not thread-safe
<code>LinkedHashMap</code>	Hash + linked list	Insertion or access	Yes	O(1)*	Basis for LRU caches
<code>TreeMap</code>	Red-black tree	Sorted by key	No null key	O(log n)	Range views

Class	Backing structure	Order	Null	Typical cost	Notes
<code>ArrayDeque</code>	Circular array	FIFO/LIFO	No	ends O(1)	Best stack/queue
<code>PriorityQueue</code>	Binary min-heap	By priority	No	offer/poll O(log n)	Iteration order is not sorted

*Amortized or average case.

15.3 Lists, Sets, Maps and Queues in Practice

```

List<String> list = new ArrayList<>(List.of("b", "a", "c"));
Collections.sort(list);                // [a, b, c]
list.add(1, "x");                      // [a, x, b, c]
list.remove("x");                     // removes by object
list.remove(0);                       // removes by index (careful with List<Integer>!)
list.removeIf(s -> s.equals("c"));

Set<Integer> set = new TreeSet<>(List.of(5, 1, 3));    // [1, 3, 5]
boolean isNew = set.add(3);                          // false (duplicate)

Map<String, Integer> stock = new HashMap<>();
stock.put("apple", 5);
stock.putIfAbsent("pear", 2);
stock.merge("apple", 3, Integer::sum);              // apple -> 8
stock.computeIfAbsent("kiwi", k -> 10);
stock.getDefault("plum", 0);
for (var e : stock.entrySet()) System.out.println(e.getKey() + "=" + e.getValue());

Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2);
System.out.println(stack.pop());                  // 2 (LIFO)
Queue<String> q = new ArrayDeque<>();
q.offer("a"); q.offer("b");
System.out.println(q.poll());                    // a (FIFO)

PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.reverseOrder()); // max-heap
pq.addAll(List.of(3, 9, 4));
System.out.println(pq.poll());                  // 9

```

15.4 HashMap Internals

- Keys are hashed (`hashCode()` spread with extra bit mixing) to choose a bucket in an array whose length is a power of two (default 16).
- Collisions are resolved with a linked list per bucket; when a bucket exceeds 8 entries and the table is large enough (64), it converts to a red-black tree for O(log n) worst-case lookups.
- When `size > capacity * loadFactor` (default 0.75) the table doubles and entries are redistributed (rehash).
- Keys should be immutable. Mutating a key's hash-relevant fields after insertion makes the entry unfindable.
- `HashMap` is not thread-safe. Use `ConcurrentHashMap` for concurrent access (fine-grained locking / CAS); `Hashtable` is legacy.

15.5 Sorting: Comparable and Comparator

```
class Employee implements Comparable<Employee> {           // natural ordering
    String name; double salary; int age;
    Employee(String n, double s, int a) { name = n; salary = s; age = a; }
    public int compareTo(Employee o) { return name.compareTo(o.name); }
    String getName() { return name; }
    double getSalary() { return salary; }
    int getAge() { return age; }
}

List<Employee> emps = new ArrayList<>();
Collections.sort(emps);                                   // uses compareTo
emps.sort(Comparator.comparingDouble(Employee::getSalary).reversed()
           .thenComparing(Employee::getName));           // custom ordering
TreeSet<Employee> byAge = new TreeSet<>(Comparator.comparingInt(Employee::getAge));
```

Pitfall: A `compare` method must never be written as `a - b` for ints (overflow). Use `Integer.compare(a, b)`.

15.6 Iteration, Fail-Fast Behavior and Immutable Collections

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4));
for (Integer n : nums) { if (n == 2) nums.remove(n); }    // ConcurrentModificationException
Iterator<Integer> it = nums.iterator();
while (it.hasNext()) if (it.next() % 2 == 0) it.remove(); // safe removal
nums.removeIf(n -> n > 2);                                // simplest

List<String> fixed = List.of("a", "b");                  // immutable; nulls not allowed
Set<String> s2 = Set.of("x", "y");
Map<String, Integer> m2 = Map.of("k", 1);                // add() -> UnsupportedOperationException
List<String> view = Collections.unmodifiableList(new ArrayList<>(fixed)); // read-only view
List<Integer> copy = List.copyOf(nums);                   // immutable copy (Java 10)
```

- **Java 21 sequenced collections:** `SequencedCollection` adds `getFirst()`, `getLast()`, `addFirst`, `addLast` and `reversed()` to lists, deques, `LinkedHashSet`, and `LinkedHashMap` (through `SequencedMap`).
- **Collections utility methods:** `sort`, `reverse`, `shuffle`, `max`, `min`, `frequency`, `unmodifiableXxx`, `synchronizedXxx`, `emptyList`, `nCopies`.
- **Choosing a collection:** need order and index access, use `ArrayList`; uniqueness, use a `Set`; key lookup, use a `Map`; sorted, use `Tree*`; FIFO/LIFO, use `ArrayDeque`; priority, use `PriorityQueue`; concurrent, use `java.util.concurrent` types.

16. Lambdas, Streams and Optional

16.1 Lambda Expressions

A lambda is a concise implementation of a functional interface (an interface with a single abstract method).

Syntax: `(parameters) -> expression` or `(parameters) -> { statements; }`. Lambdas can capture local variables that are **effectively final**.

```
Runnable r = () -> System.out.println("run");
Comparator<String> byLen = (a, b) -> Integer.compare(a.length(), b.length());
BiFunction<Integer, Integer, Integer> add = (x, y) -> x + y;
int base = 10; // effectively final
Function<Integer, Integer> plusBase = n -> n + base;
// base++; // would make the capture illegal
```

16.2 Built-in Functional Interfaces (java.util.function)

Interface	Method	Purpose	Example
<code>Function<T,R></code>	<code>R apply(T)</code>	Transform	<code>s -> s.length()</code>
<code>BiFunction<T,U,R></code>	<code>R apply(T,U)</code>	Transform two inputs	<code>(a, b) -> a + b</code>
<code>UnaryOperator<T></code>	<code>T apply(T)</code>	Same-type transform	<code>String::trim</code>
<code>BinaryOperator<T></code>	<code>T apply(T,T)</code>	Combine	<code>Integer::sum</code>
<code>Predicate<T></code>	<code>boolean test(T)</code>	Condition	<code>n -> n > 0</code>
<code>Consumer<T></code>	<code>void accept(T)</code>	Side effect	<code>System.out::println</code>
<code>Supplier<T></code>	<code>T get()</code>	Produce a value	<code>ArrayList::new</code>
<code>Runnable / Callable<V></code>	<code>void run() / V call()</code>	Tasks (Callable can throw)	<code>() -> compute()</code>

Primitive specializations avoid boxing: `IntFunction`, `IntPredicate`, `ToIntFunction`, `IntUnaryOperator`, `IntBinaryOperator`, etc. Compose with `andThen`, `compose`, `and`, `or`, `negate`.

16.3 Method References

Kind	Syntax	Equivalent lambda
Static method	<code>Integer::parseInt</code>	<code>s -> Integer.parseInt(s)</code>
Bound instance	<code>System.out::println</code>	<code>x -> System.out.println(x)</code>
Unbound instance	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>
Constructor	<code>ArrayList::new</code>	<code>() -> new ArrayList<>()</code>

16.4 The Streams API

A stream is a **lazy pipeline** over a data source: **source** -> **intermediate operations** -> **terminal operation**.

Nothing executes until the terminal operation runs. Streams do not modify the source and can be consumed only once.

Type	Operations
Sources	<code>collection.stream()</code> , <code>Arrays.stream(arr)</code> , <code>Stream.of(...)</code> , <code>IntStream.range(0, n)</code> , <code>Stream.iterate</code> , <code>Stream.generate</code> , <code>Files.lines(path)</code>
Intermediate	<code>filter</code> , <code>map</code> , <code>flatMap</code> , <code>distinct</code> , <code>sorted</code> , <code>limit</code> , <code>skip</code> , <code>peek</code> , <code>mapToInt</code> , <code>takeWhile</code> , <code>dropWhile</code>
Terminal	<code>collect</code> , <code>forEach</code> , <code>reduce</code> , <code>count</code> , <code>min</code> , <code>max</code> , <code>sum</code> , <code>anyMatch</code> , <code>allMatch</code> , <code>noneMatch</code> , <code>findFirst</code> , <code>findAny</code> , <code>toList</code> , <code>toArray</code>

```
record Emp(String name, String dept, double salary) {}
List<Emp> emps = List.of(new Emp("Asha", "IT", 90), new Emp("Ravi", "IT", 70),
                        new Emp("Meera", "HR", 60), new Emp("John", "HR", 85));

List<String> highPaid = emps.stream()
    .filter(e -> e.salary() > 65)
    .sorted(Comparator.comparingDouble(Emp::salary).reversed())
    .map(Emp::name)
    .toList();                                     // [Asha, John, Ravi]

Map<String, List<Emp>> byDept = emps.stream().collect(Collectors.groupingBy(Emp::dept));
Map<String, Double> avg = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept, Collectors.averagingDouble(Emp::salary)));
Map<Boolean, Long> split = emps.stream()
    .collect(Collectors.partitioningBy(e -> e.salary() >= 80, Collectors.counting()));
String names = emps.stream().map(Emp::name).collect(Collectors.joining(", ", "[", "]"));
double total = emps.stream().mapToDouble(Emp::salary).sum();           // 305.0
Optional<Emp> top = emps.stream().max(Comparator.comparingDouble(Emp::salary));
int sumSquares = IntStream.rangeClosed(1, 5).map(x -> x * x).sum();    // 55
List<Integer> flat = Stream.of(List.of(1, 2), List.of(3))
    .flatMap(List::stream).toList();                                   // [1, 2, 3]
```

- **Collectors** include `toList`, `toSet`, `toMap`, `groupingBy`, `partitioningBy`, `joining`, `counting`, `summingInt`, `averagingInt`, `summarizingInt`, `mapping`.
- `parallelStream()` splits work across the common ForkJoin pool. Use it only for large, CPU-bound, stateless, side-effect-free work and measure first.
- Avoid stateful lambdas and modifying external state from stream operations.

16.5 Optional

Optional<T> models a value that may be absent, making "no result" explicit in the return type. Use it for return values, not for fields or parameters.

```
Optional<String> maybe = Optional.ofNullable(findName());
String upper = maybe.map(String::toUpperCase).orElse("UNKNOWN");
maybe.ifPresent(System.out::println);
maybe.ifPresentOrElse(n -> System.out.println(n), () -> System.out.println("none"));
String must = maybe.orElseThrow(() -> new NoSuchElementException("no name"));
Optional<Integer> len = maybe.filter(s -> !s.isBlank()).map(String::length);
// avoid: maybe.get() without isPresent(); prefer orElse / orElseThrow
```

17. Essential Utility APIs

17.1 Math, Random and Numbers

```
Math.abs(-5); Math.pow(2, 10);           // 5, 1024.0
Math.sqrt(16); Math.cbrt(27);           // 4.0, 3.0
Math.max(3, 9); Math.min(3, 9);
Math.floor(2.7); Math.ceil(2.1); Math.round(2.5); // 2.0, 3.0, 3
Math.floorMod(-7, 3);                   // 2 (always non-negative for positive divisor)
Math.addExact(Integer.MAX_VALUE, 1);     // throws ArithmeticException (overflow detected)

java.util.concurrent.ThreadLocalRandom.current().nextInt(1, 7); // dice roll, thread-friendly
new java.util.Random(42).nextInt(100); // seeded: reproducible sequence
Integer.MAX_VALUE; Long.MIN_VALUE; Double.compare(1.0, 2.0);
Integer.toBinaryString(10);              // "1010"
Integer.parseInt("ff", 16);              // 255
```

17.2 BigInteger and BigDecimal

```
import java.math.*;

BigInteger big = new BigInteger("123456789012345678901234567890");
BigInteger sq = big.multiply(big);

BigDecimal a = new BigDecimal("0.10");
BigDecimal b = new BigDecimal("0.20");
System.out.println(a.add(b)); // 0.30 (exact)
BigDecimal third = new BigDecimal("10").divide(new BigDecimal("3"), 4, RoundingMode.HALF_UP);
System.out.println(third); // 3.3333
```

Tip: Construct `BigDecimal` from a String, not a `double`: `new BigDecimal(0.1)` carries the binary error of 0.1.

17.3 Date and Time API (java.time, Java 8+)

The `java.time` classes are **immutable** and **thread-safe**, replacing the error-prone `Date` and `Calendar`.

Class	Represents
<code>LocalDate</code> / <code>LocalTime</code> / <code>LocalDateTime</code>	Date, time, or both without a time zone
<code>ZonedDateTime</code> / <code>OffsetDateTime</code>	Date-time with a zone or offset
<code>Instant</code>	A point on the UTC timeline (machine time)
<code>Duration</code> / <code>Period</code>	Time-based amount (seconds) / date-based amount (years, months, days)
<code>DateTimeFormatter</code>	Formatting and parsing

```
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

LocalDate today = LocalDate.now();
LocalDate d = LocalDate.of(2026, 9, 19);
LocalDate next = d.plusDays(30).withDayOfMonth(1);
long days = ChronoUnit.DAYS.between(d, LocalDate.of(2026, 12, 31)); // 103
Period age = Period.between(LocalDate.of(2003, 5, 14), d); // P23Y4M5D
DayOfWeek dow = d.getDayOfWeek(); // SATURDAY
```

```

LocalDateTime meeting = LocalDateTime.of(2026, 10, 1, 14, 30);
ZonedDateTime india = meeting.atZone(ZoneId.of("Asia/Kolkata"));
ZonedDateTime tokyo = india.withZoneSameInstant(ZoneId.of("Asia/Tokyo"));
String txt = meeting.format(DateTimeFormatter.ofPattern("dd MMM yyyy, hh:mm a"));
LocalDate parsed = LocalDate.parse("2026-12-25");
Instant start = Instant.parse("2026-01-01T00:00:00Z");
Duration dur = Duration.between(start, start.plusSeconds(5400));           // PT1H30M

```

17.4 Other Useful Classes

- **Objects:** `equals`, `hash`, `requireNonNull`, `toString`, `isNull`. `UUID.randomUUID()` creates random identifiers.
- **StringJoiner**, **Locale**, **Currency**, **NumberFormat** for formatting; **Base64**, **MessageDigest** for encoding and hashing.
- **System:** `currentTimeMillis`, `nanoTime` (elapsed-time measurement), `getProperty`, `getenv`, `arraycopy`, `exit`.
- `Runtime.getRuntime()`: `availableProcessors()`, `totalMemory()`, `freeMemory()`, `maxMemory()`.
- `java.util.Scanner`, `Properties`, `BitSet`, `Timer`, `Locale`, `ResourceBundle` (internationalization).

18. Input/Output and Files

18.1 Streams: Bytes vs Characters

	Byte streams (binary)	Character streams (text)
Base classes	<code>InputStream</code> / <code>OutputStream</code>	<code>Reader</code> / <code>Writer</code>
Files	<code>FileInputStream</code> , <code>FileOutputStream</code>	<code>FileReader</code> , <code>FileWriter</code>
Buffering	<code>BufferedInputStream</code> / <code>OutputStream</code>	<code>BufferedReader</code> , <code>BufferedWriter</code>
Bridge	<code>InputStreamReader</code> / <code>OutputStreamWriter</code> convert bytes to chars using a charset	
Convenience	<code>DataInputStream</code> , <code>ObjectInputStream</code>	<code>PrintWriter</code> , <code>Scanner</code>

18.2 Modern File Handling with NIO.2

```

import java.nio.file.*;
import java.nio.charset.StandardCharsets;
import java.io.IOException;

Path dir = Path.of("data");
Files.createDirectories(dir);
Path file = dir.resolve("notes.txt");

Files.writeString(file, "Line 1\nLine 2\n");           // Java 11
Files.write(file, List.of("Line 3"), StandardOpenOption.APPEND);
String all = Files.readString(file);
List<String> lines = Files.readAllLines(file, StandardCharsets.UTF_8);

try (Stream<String> s = Files.lines(file)) {             // lazy, must be closed
    s.filter(l -> l.contains("2")).forEach(System.out::println);
}
try (Stream<Path> walk = Files.walk(dir)) {              // recursive listing
    walk.filter(Files::isRegularFile).forEach(System.out::println);
}

```

```

}
Files.copy(file, dir.resolve("copy.txt"), StandardCopyOption.REPLACE_EXISTING);
Files.deleteIfExists(dir.resolve("copy.txt"));
System.out.println(Files.exists(file) + " " + Files.size(file));

```

18.3 Classic java.io Example

```

try (BufferedReader br = new BufferedReader(new FileReader("input.txt"));
     BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        bw.write(line.trim());
        bw.newLine();
    }
} catch (IOException e) {
    System.err.println("I/O problem: " + e.getMessage());
}

```

18.4 Serialization

Serialization converts an object graph to bytes. The class must implement `Serializable`; fields marked `transient` and `static` fields are skipped. Declare `serialVersionUID` to control version compatibility.

```

class User implements java.io.Serializable {
    private static final long serialVersionUID = 1L;
    String name;
    transient String password;           // not saved
}

try (var out = new ObjectOutputStream(new FileOutputStream("user.bin"))) {
    out.writeObject(user);
}
try (var in = new ObjectInputStream(new FileInputStream("user.bin"))) {
    User u = (User) in.readObject();
}

```

Warning: Java's built-in deserialization of untrusted data is a well-known security risk. Prefer JSON or protobuf and never deserialize data you do not trust.

18.5 NIO Channels and Buffers

NIO offers `ByteBuffer`, `FileChannel`, memory-mapped files and non-blocking `Selector`-based network I/O for high-throughput servers. Most applications use `Files` for files and a framework (Netty, Spring WebFlux) for scalable networking.

19. Multithreading and Concurrency

A **process** has its own memory; **threads** within a process share the heap and run concurrently. Concurrency improves throughput and responsiveness but introduces hazards: race conditions, visibility problems and deadlocks.

19.1 Creating and Managing Threads

```
class Worker extends Thread { // option 1: extend Thread (avoid)
    public void run() { System.out.println("in " + getName()); }
}
Thread t1 = new Thread(() -> System.out.println("lambda task"), "worker-1"); // Runnable
t1.start(); // start() creates a new thread; calling run() would not
t1.join(); // wait for completion
Thread.sleep(200); // pause the current thread
t1.interrupt(); // cooperative cancellation request
Thread daemon = new Thread(() -> {});
daemon.setDaemon(true); // does not keep the JVM alive
```

State	Meaning
NEW	Created, not started
RUNNABLE	Running or ready to run
BLOCKED	Waiting to acquire a monitor lock
WAITING / TIMED_WAITING	Waiting for another thread (wait , join , park) or for a timeout (sleep)
TERMINATED	Finished

19.2 Race Conditions and synchronized

```
class Counter {
    private int count;
    public synchronized void inc() { count++; } // locks on `this`
    public int get() { synchronized (this) { return count; } }
}
// count++ is read-modify-write, NOT atomic. Without synchronization, updates are lost.

class Cache {
    private static final Object LOCK = new Object(); // safer than `this`
    static void update() { synchronized (LOCK) { /* critical section */ } }
}
```

- **synchronized** provides **mutual exclusion** and **visibility**: changes made before releasing a lock are visible to the next thread acquiring it. Locks are **reentrant**.
- **volatile** guarantees visibility and ordering of a single variable but not atomic compound actions like **count++**.
- The **Java Memory Model** defines **happens-before** rules (monitor unlock/lock, volatile write/read, **Thread.start/join**) that make shared data safe to read.
- **Atomics** (**AtomicInteger**, **AtomicLong**, **AtomicReference**, **LongAdder**) use compare-and-swap for lock-free updates.

19.3 wait, notify and Producer-Consumer

```
class Buffer {
    private final Queue<Integer> q = new LinkedList<>();
    private final int cap = 5;

    synchronized void put(int v) throws InterruptedException {
        while (q.size() == cap) wait();           // always wait in a loop
        q.add(v);
        notifyAll();
    }
    synchronized int take() throws InterruptedException {
        while (q.isEmpty()) wait();
        int v = q.remove();
        notifyAll();
        return v;
    }
}
// In practice, prefer BlockingQueue: put() and take() block automatically.
BlockingQueue<Integer> bq = new ArrayBlockingQueue<>(5);
```

19.4 Locks

```
import java.util.concurrent.locks.*;

class Account {
    private final Lock lock = new ReentrantLock();
    private double balance;
    void deposit(double amt) {
        lock.lock();
        try { balance += amt; }
        finally { lock.unlock(); }                // always release in finally
    }
    boolean tryDeposit(double amt) throws InterruptedException {
        if (lock.tryLock(1, TimeUnit.SECONDS)) { // timed acquisition
            try { balance += amt; return true; } finally { lock.unlock(); }
        }
        return false;
    }
}
// ReadWriteLock: many concurrent readers or one writer. StampedLock: optimistic reads.
```

19.5 Executors and Thread Pools

Factory method	Behavior
<code>newFixedThreadPool(n)</code>	n threads, unbounded queue; predictable resource use
<code>newCachedThreadPool()</code>	Creates threads as needed, reuses idle ones; can grow without bound
<code>newSingleThreadExecutor()</code>	Tasks run sequentially in one thread
<code>newScheduledThreadPool(n)</code>	Delayed and periodic tasks
<code>newVirtualThreadPerTaskExecutor()</code>	One virtual thread per task (Java 21)
<code>ForkJoinPool / newWorkStealingPool()</code>	Work-stealing pool for divide-and-conquer and parallel streams

```
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> {           // Callable returns a value
    Thread.sleep(100);
    return 42;
});
```

```

});
System.out.println(f.get(1, TimeUnit.SECONDS)); // blocks; ExecutionException on failure

List<Callable<String>> jobs = List.of(() -> "a", () -> "b");
for (Future<String> r : pool.invokeAll(jobs)) System.out.println(r.get());

pool.shutdown(); // finish queued tasks, accept no more
if (!pool.awaitTermination(5, TimeUnit.SECONDS)) pool.shutdownNow();

// Java 19+: ExecutorService is AutoCloseable
try (var ex = Executors.newFixedThreadPool(2)) {
    ex.submit(() -> System.out.println("task"));
}

```

19.6 CompletableFuture

```

CompletableFuture<String> cf = CompletableFuture
    .supplyAsync(() -> fetchUser(7)) // ForkJoinPool.commonPool()
    .thenApply(user -> user.toUpperCase()) // transform
    .thenCombine(CompletableFuture.supplyAsync(() -> "!"), String::concat) // combine
    .exceptionally(ex -> "fallback") // recover from errors
    .whenComplete((res, ex) -> System.out.println("done: " + res));

CompletableFuture.allOf(cf, cf).join(); // wait for many

```

19.7 Synchronizers and Concurrent Collections

Utility	Use
<code>CountDownLatch</code>	One-time gate: wait until N events have happened
<code>CyclicBarrier</code>	Threads wait for each other at a repeating barrier
<code>Semaphore</code>	Limit concurrent access to N permits
<code>Phaser</code> , <code>Exchanger</code>	Multi-phase coordination; swapping data between two threads
<code>ConcurrentHashMap</code>	Scalable thread-safe map; atomic <code>merge</code> , <code>compute</code> , <code>putIfAbsent</code>
<code>CopyOnWriteArrayList</code>	Thread-safe list optimized for many reads, few writes
<code>BlockingQueue</code> (<code>ArrayBlockingQueue</code> , <code>LinkedBlockingQueue</code>)	Producer-consumer hand-off
<code>ConcurrentLinkedQueue</code> , <code>ConcurrentSkipListMap</code>	Lock-free queue; sorted concurrent map

```

ConcurrentHashMap<String, LongAdder> hits = new ConcurrentHashMap<>();
hits.computeIfAbsent("home", k -> new LongAdder()).increment();

CountDownLatch ready = new CountDownLatch(3);
for (int i = 0; i < 3; i++) new Thread(() -> { /* init */ ready.countDown(); }).start();
ready.await(); // proceeds when count reaches zero

```

19.8 Deadlock, Starvation and Livelock

Deadlock needs four conditions (Coffman): mutual exclusion, hold-and-wait, no preemption, and circular wait. Break any one to prevent it.

```

// Thread A: lock(x), lock(y)    Thread B: lock(y), lock(x) -> possible deadlock
// Fix: always acquire locks in one global order, or use tryLock with timeout.

```

- **Starvation:** a thread never gets CPU or a lock (unfair scheduling). **Livelock:** threads keep reacting to each other without progress.
- Diagnose with a thread dump: `jstack <pid>` or `jcmd <pid> Thread.print`; the JVM reports detected deadlocks.
- **ThreadLocal** gives each thread its own copy of a variable; always call `remove()` in pooled threads to avoid leaks.

19.9 Virtual Threads (Java 21)

Virtual threads are lightweight threads scheduled by the JVM onto a small pool of carrier (platform) threads. They make the simple **thread-per-request** blocking style scale to hundreds of thousands of concurrent tasks. When a virtual thread blocks on I/O, it is unmounted and the carrier thread runs other work.

```
Thread.startVirtualThread(() -> System.out.println("virtual: " + Thread.currentThread()));

try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        int id = i;
        executor.submit(() -> { Thread.sleep(1000); return id; }); // 10,000 blocking tasks
    }
} // close() waits for all tasks
```

Tip: Do not pool virtual threads, and on Java 21 to 23, avoid holding `synchronized` locks around blocking calls in code that must scale (pinning); prefer `ReentrantLock` there. Virtual threads help with I/O-bound work, not CPU-bound work.

20. JVM Internals, Memory and Performance

20.1 Memory Areas

Area	Contents	Scope	Typical error
Heap	All objects and arrays; Young (Eden + 2 Survivor) and Old generations	Shared	<code>OutOfMemoryError: Java heap space</code>
Metaspace	Class metadata (native memory)	Shared	<code>OutOfMemoryError: Metaspace</code>
Thread stack	Frames: locals, operand stack, return address	Per thread	<code>StackOverflowError</code>
PC register / native stack	Current instruction; native method frames	Per thread	
Code cache	JIT-compiled native code	Shared	Code cache full warning
Direct memory	Off-heap buffers (NIO, native libs)	Shared	<code>OutOfMemoryError: Direct buffer memory</code>

20.2 Garbage Collection

- An object is eligible for collection when it is **unreachable** from any GC root (thread stacks, static fields, JNI references).
- **Generational hypothesis:** most objects die young. New objects go to Eden; a *minor GC* copies survivors between Survivor spaces and promotes long-lived objects to the Old generation; a *major/mixed GC* cleans

the Old generation.

- **System.gc()** is only a hint. Do not rely on it.
- **Reference strengths:** strong (normal), **SoftReference** (cleared under memory pressure; caches), **WeakReference** (cleared at the next GC; **WeakHashMap**), **PhantomReference** (post-mortem cleanup).

Collector	Flag	Characteristics
Serial	-XX:+UseSerialGC	Single-threaded; tiny heaps and small containers
Parallel	-XX:+UseParallelGC	Throughput-oriented; multiple GC threads; longer pauses
G1 (default since Java 9)	-XX:+UseG1GC	Region-based; balanced throughput and pause goals (-XX:MaxGCPauseMillis)
ZGC	-XX:+UseZGC	Concurrent, very low pauses (typically sub-millisecond), large heaps; generational mode in Java 21+
Shenandoah	-XX:+UseShenandoahGC	Low-pause concurrent collector (available in many OpenJDK builds)

20.3 Useful JVM Options and Tools

```
java -Xms512m -Xmx2g -Xss512k -XX:+UseG1GC -jar app.jar
java -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/dump.hprof -jar app.jar
java -Xlog:gc*:file=gc.log -jar app.jar           # unified GC logging (Java 9+)
java -XX:+PrintFlagsFinal -version | grep -i heap
jcmd <pid> GC.heap_info          jcmd <pid> Thread.print          jcmd <pid> VM.flags
```

Tool	Purpose
jps, jcmd	List JVM processes and send diagnostic commands
jstack	Thread dumps (deadlocks, blocked threads)
jmap, jcmd GC.heap_dump	Heap histograms and heap dumps (analyze in Eclipse MAT or VisualVM)
jstat	Live GC statistics
Java Flight Recorder (JFR) + JDK Mission Control	Low-overhead production profiling and event recording
jshell, jlink, jpackage, jdeps	REPL, custom runtime images, native installers, dependency analysis

20.4 Memory Leaks and Performance Tips

- Typical leaks: static collections that only grow, unbounded caches, unregistered listeners/callbacks, unclosed resources, **ThreadLocal** in thread pools, classloader leaks.
- Measure before optimizing: use JFR or a profiler and a benchmarking harness such as **JMH**; micro-benchmarks written by hand are usually wrong (warm-up, dead-code elimination).
- Prefer the right algorithm and data structure first. Reduce allocation in hot paths; reuse **StringBuilder**; avoid unnecessary boxing (**IntStream**, primitive arrays).
- Size collections when the size is known (**new ArrayList<>(n)**, **new HashMap<>(capacity)**).
- Compile-time constant strings are interned; runtime concatenation in loops is costly.

20.5 Reflection and Annotations

```
Class<?> c = Class.forName("java.util.ArrayList");
Object list = c.getDeclaredConstructor().newInstance();
Method add = c.getMethod("add", Object.class);
add.invoke(list, "reflected");
for (Field f : String.class.getDeclaredFields()) System.out.println(f.getName());

// Custom annotation with runtime retention
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Timed { String value() default ""; }

class Service {
    @Timed("load") public void load() { }
}
for (Method m : Service.class.getDeclaredMethods()) {
    if (m.isAnnotationPresent(Timed.class)) System.out.println(m.getName());
}
```

Reflection powers frameworks (Spring dependency injection, Hibernate mapping, JUnit discovery, Jackson serialization). It bypasses compile-time checks and is slower, and strong encapsulation of JDK internals (JPMS) restricts deep reflection on JDK classes. Built-in annotations: `@Override`, `@Deprecated`, `@SuppressWarnings`, `@FunctionalInterface`, `@SafeVarargs`.

20.6 Modules (JPMS)

```
// module-info.java at the root of a module
module com.shop.billing {
    requires java.sql;                // dependency
    requires transitive com.shop.core; // re-exported to dependents
    exports com.shop.billing.api;     // public API packages
    opens com.shop.billing.model to com.fasterxml.jackson.databind; // reflection access
}
```

Modules give strong encapsulation and reliable configuration. Many applications still run on the classpath; `jlink` can build a minimal runtime from the modules you need.

21. Networking, Databases, Testing and Logging

21.1 Networking

```
// TCP echo server
try (ServerSocket server = new ServerSocket(5000)) {
    while (true) {
        Socket client = server.accept();
        Thread.startVirtualThread(() -> {
            try (client;
                var in = new BufferedReader(new InputStreamReader(client.getInputStream()));
                var out = new PrintWriter(client.getOutputStream(), true)) {
                String line;
                while ((line = in.readLine()) != null) out.println("echo: " + line);
            } catch (IOException ignored) { }
        });
    }
}

// HTTP client (Java 11+)
HttpClient http = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(URI.create("https://api.example.com/items"))
```

```

        .header("Accept", "application/json")
        .timeout(Duration.ofSeconds(10))
        .GET()
        .build();
    HttpResponse<String> res = http.send(req, HttpResponse.BodyHandlers.ofString());
    System.out.println(res.statusCode() + " " + res.body());
    var async = http.sendAsync(req, HttpResponse.BodyHandlers.ofString());    // non-blocking

```

21.2 JDBC

JDBC is the standard API to relational databases. The flow is: obtain a **Connection** (through **DriverManager** or a **DataSource**), create a **PreparedStatement**, execute, process the **ResultSet**, and close everything (try-with-resources).

```

String url = "jdbc:postgresql://localhost:5432/shop";
try (Connection con = DriverManager.getConnection(url, "user", "secret")) {
    con.setAutoCommit(false);                                // manual transaction
    try (PreparedStatement ins = con.prepareStatement(
        "INSERT INTO orders(customer_id, total) VALUES (?, ?)")) {
        ins.setInt(1, 7);
        ins.setBigDecimal(2, new BigDecimal("199.99"));
        ins.executeUpdate();
    }
    String sql = "SELECT id, total FROM orders WHERE customer_id = ?";
    try (PreparedStatement q = con.prepareStatement(sql)) {
        q.setInt(1, 7);
        try (ResultSet rs = q.executeQuery()) {
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " " + rs.getBigDecimal("total"));
            }
        }
    }
    con.commit();
} catch (SQLException e) {
    // con.rollback() when a transaction fails
}

```

- Always use **PreparedStatement** with parameters. String-concatenated SQL enables SQL injection.
- Opening connections is expensive: use a **connection pool** (HikariCP is the common choice).
- Use batches (**addBatch/executeBatch**) for bulk inserts. Use **BigDecimal** for money columns.
- In real projects, JDBC is usually wrapped by **JPA/Hibernate**, **Spring Data**, **jOOQ** or **MyBatis**.

21.3 JSON, Testing and Logging

```

// JSON with Jackson
ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(new Point(1, 2));    // {"x":1,"y":2}
Point p = mapper.readValue(json, Point.class);

// JUnit 5 + Mockito
class CalculatorTest {
    @Test void addsNumbers() { assertEquals(5, new Calculator().add(2, 3)); }

    @Test void rejectsNegative() {
        assertThrows(IllegalArgumentException.class, () -> new Calculator().sqrt(-1));
    }

    @Test void usesRepository() {
        UserRepo repo = mock(UserRepo.class);
        when(repo.find(1)).thenReturn(Optional.of(new User("Asha")));
        assertEquals("Asha", new UserService(repo).name(1));
    }
}

```

```

        verify(repo).find(1);
    }
}

// Logging with SLF4J (facade) + Logback/Log4j2 (implementation)
private static final Logger log = LoggerFactory.getLogger(OrderService.class);
log.info("Order {} created for user {}", orderId, userId);        // parameterized, no string concat
log.error("Payment failed", exception);                          // exception goes last

```

- Testing pyramid: many fast **unit tests** (JUnit 5, AssertJ, Mockito), fewer **integration tests** (Testcontainers), and a few end-to-end tests.
- Use log levels correctly: **ERROR**, **WARN**, **INFO**, **DEBUG**, **TRACE**. Never log secrets or personal data. Avoid **System.out.println** in production code.

22. The Java Ecosystem and Version Highlights

22.1 Build Tools and Dependencies

```

<!-- Maven: pom.xml -->
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0.0</version>
  <properties><maven.compiler.release>21</maven.compiler.release></properties>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>

// Gradle (Kotlin DSL): build.gradle.kts
plugins { java }
repositories { mavenCentral() }
dependencies { testImplementation("org.junit.jupiter:junit-jupiter:5.10.2") }
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }

```

- **Maven:** convention-over-configuration, XML, lifecycle **validate**, **compile**, **test**, **package**, **verify**, **install**, **deploy**. Commands: **mvn clean package**, **mvn test**.
- **Gradle:** flexible, faster incremental builds, Groovy/Kotlin DSL. Commands: **./gradlew build**, **./gradlew test**.
- Artifacts come from **Maven Central**. Coordinates are **groupId:artifactId:version**.

22.2 Frameworks and Platforms

Area	Technologies	What they give you
Web / REST backends	Spring Boot , Quarkus, Micronaut, Jakarta EE	Dependency injection, embedded server, REST, security, configuration
Persistence	Hibernate/JPA, Spring Data, jOOQ, MyBatis	Object-relational mapping and repositories
Messaging / streaming	Kafka clients, RabbitMQ, JMS	Asynchronous, event-driven systems

Area	Technologies	What they give you
Reactive	Project Reactor, WebFlux, RxJava, Vert.x	Non-blocking pipelines
Testing	JUnit 5, Mockito, AssertJ, Testcontainers, Selenium	Unit, integration and UI tests
Desktop / mobile	JavaFX, Swing, Android (Java/Kotlin)	GUI and mobile apps
Big data	Hadoop, Spark, Flink, Kafka Streams	Distributed data processing
Cloud / DevOps	Docker, Kubernetes, GraalVM native image, CI/CD	Packaging, deployment, fast startup

22.3 A Minimal Spring Boot REST Service

```

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}

@Entity
class Book {
    @Id @GeneratedValue private Long id;
    private String title;
    protected Book() {}
    Book(String title) { this.title = title; }
    public Long getId() { return id; }
    public String getTitle() { return title; }
}

interface BookRepository extends JpaRepository<Book, Long> {
    List<Book> findByTitleContainingIgnoreCase(String text);    // derived query
}

@RestController
@RequestMapping("/api/books")
class BookController {
    private final BookRepository repo;
    BookController(BookRepository repo) { this.repo = repo; }    // constructor injection

    @GetMapping                public List<Book> all() { return repo.findAll(); }
    @GetMapping("/{id}")       public Book one(@PathVariable Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
    }
    @PostMapping                public Book create(@RequestBody Book b) { return repo.save(b); }
}

```

22.4 Feature Timeline

Version	Notable features
Java 5	Generics, annotations, enums, autoboxing, enhanced for, varargs, <code>java.util.concurrent</code>
Java 7	try-with-resources, diamond <code><></code> , strings in <code>switch</code> , NIO.2, multi-catch
Java 8 (LTS)	Lambdas, Streams, <code>Optional</code> , default methods, <code>java.time</code> , method references
Java 9	Modules (JPMS), <code>List.of/Set.of/Map.of</code> , <code>jshell</code> , private interface methods, G1 default

Version	Notable features
Java 10 / 11 (LTS)	<code>var</code> ; <code>HttpClient</code> , <code>String</code> helpers (<code>isBlank</code> , <code>strip</code> , <code>lines</code> , <code>repeat</code>), <code>Files.readString</code> , single-file launch
Java 14 to 16	Switch expressions, text blocks, pattern matching for <code>instanceof</code> , records
Java 17 (LTS)	Sealed classes, strong encapsulation of JDK internals, <code>RandomGenerator</code>
Java 21 (LTS)	Virtual threads, pattern matching for <code>switch</code> , record patterns, sequenced collections, generational ZGC
Java 22 to 24	Unnamed variables and patterns (<code>_</code>), Foreign Function and Memory API, stream gatherers
Java 25 (LTS)	Latest LTS at the time of writing; finalizes features such as flexible constructor bodies, module import declarations and scoped values

Note: Feature availability changes with each release. Always check the official OpenJDK release notes and JEP list for the exact version you deploy.

23. Best Practices and Common Pitfalls

23.1 Coding Best Practices

- **Readable code first:** meaningful names, short methods (one purpose), small classes, consistent formatting. Follow a style guide (Google Java Style or your team's).
- **Prefer immutability:** `final` fields, immutable value objects, `List.of`, records. Fewer states means fewer bugs.
- **Program to interfaces:** declare `List<String> x = new ArrayList<>()`, not `ArrayList<String> x`.
- **Fail fast:** validate inputs at the boundary with `Objects.requireNonNull` and precise exceptions.
- **Override `equals`, `hashCode` and `toString` together** for value-like classes (or use records).
- **Close resources** with try-with-resources. **Do not return null** for collections; return empty ones. Use `Optional` for absent single values.
- **Use the standard library:** `java.time`, `Files`, collections, streams, `Objects`, `Math.floorMod`. Avoid reinventing them.
- **Write tests** and run them in CI. Keep methods testable by injecting dependencies.
- **Document intent** (why), not mechanics (what). Use Javadoc for public APIs.

23.2 Frequent Pitfalls

Pitfall	Why it bites	Fix
<code>==</code> on Strings/wrappers	Compares references; works by accident for pooled/cached values	Use <code>equals</code> (or <code>Objects.equals</code>)
Integer overflow	<code>int</code> wraps silently	Use <code>long</code> , <code>Math.addExact</code> , <code>BigInteger</code>
<code>float</code> / <code>double</code> for money	Binary rounding errors	<code>BigDecimal</code> from String
Mutating a collection while iterating	<code>ConcurrentModificationException</code>	<code>Iterator.remove</code> , <code>removeIf</code> , copy

Pitfall	Why it bites	Fix
Mutable keys in HashMap/HashSet	Hash changes; entry is lost	Use immutable keys
Overriding <code>equals</code> without <code>hashCode</code>	Breaks hash collections	Override both
Catching <code>Exception</code> and ignoring it	Hides bugs	Handle specifically or log and rethrow
String + in loops	O(n^2) copying	<code>StringBuilder</code>
Forgetting <code>break</code> in classic <code>switch</code>	Fall-through	Use arrow-form switch
Non-thread-safe shared state	Races and lost updates	Immutability, locks, concurrent classes
<code>Thread.run()</code> instead of <code>start()</code>	Runs on the caller thread	Call <code>start()</code>
<code>NullPointerException</code>	Null dereference	<code>Optional</code> , null checks, <code>requireNonNull</code> , annotations
Resource leaks	Unclosed streams/connections	try-with-resources, pools
Swallowing <code>InterruptedException</code>	Breaks cancellation	Restore interrupt flag or rethrow

23.3 Security Basics

- Validate and sanitize all external input. Use **prepared statements** for SQL; encode output to prevent XSS.
- Never hard-code secrets. Use environment variables or a secrets manager. Do not log passwords or tokens.
- Store passwords with a slow salted hash (bcrypt, scrypt, Argon2, PBKDF2), never plain text or plain SHA.
- Use `SecureRandom` for security-sensitive randomness. Avoid Java native deserialization of untrusted data.
- Keep the JDK and dependencies patched (Dependabot, OWASP Dependency-Check). Use TLS for all network traffic.

24. Frequently Asked Interview Questions

Q1. Why is Java platform independent?

Source compiles to bytecode, which any JVM implementation can run. The JVM is platform specific; the bytecode is not.

Q2. Difference between JDK, JRE and JVM?

JVM runs bytecode; JRE = JVM + libraries; JDK = JRE + development tools (javac, jar, jshell).

Q3. `==` vs `equals()`?

`==` compares primitives by value and objects by reference. `equals()` compares logical content when overridden.

Q4. Why are Strings immutable?

Security, thread safety, string pooling, and cached hash codes that make them safe hash-map keys.

Q5. String vs StringBuilder vs StringBuffer?

String is immutable; StringBuilder is mutable and fast; StringBuffer is mutable and synchronized.

Q6. Abstract class vs interface?

Abstract classes can hold state and constructors and are single-inherited; interfaces define contracts, allow multiple implementation, and since Java 8 can have default methods.

Q7. Overloading vs overriding?

Overloading: same name, different parameters, resolved at compile time. Overriding: subclass redefines an inherited method, resolved at runtime.

Q8. Can we override static or private methods?

No. Static methods are hidden; private methods are not inherited.

Q9. What is the contract between equals() and hashCode()?

Equal objects must have equal hash codes. Not vice versa. Override both.

Q10. Checked vs unchecked exceptions?

Checked exceptions must be handled or declared; unchecked (RuntimeException, Error) need not be.

Q11. final, finally, finalize?

final: constant/non-overridable/non-extendable. **finally**: cleanup block. **finalize()**: deprecated GC hook.

Q12. How does HashMap work?

Hash of the key selects a bucket; equals resolves collisions; buckets become trees beyond 8 entries; resizes at 0.75 load factor.

Q13. HashMap vs ConcurrentHashMap vs Hashtable?

HashMap is not thread-safe and allows one null key. Hashtable locks the whole table (legacy). ConcurrentHashMap gives scalable thread safety and disallows nulls.

Q14. ArrayList vs LinkedList?

ArrayList has O(1) random access and is cache friendly; LinkedList has O(1) insertion at known nodes but O(n) access. ArrayList is the default choice.

Q15. Comparable vs Comparator?

Comparable defines a class's natural order (**compareTo**); Comparator defines external, multiple orderings (**compare**).

Q16. What is the difference between fail-fast and fail-safe iterators?

Fail-fast throws ConcurrentModificationException on structural change (ArrayList, HashMap). Fail-safe iterators work on a snapshot or weakly consistent view (CopyOnWriteArrayList, ConcurrentHashMap).

Q17. Explain the Java memory model areas.

Heap (objects, GC-managed), stack (per-thread frames), metaspace (class metadata), plus PC registers and native stacks.

Q18. How does garbage collection work?

Unreachable objects are reclaimed. Generational collectors copy young survivors and promote old ones; G1 is the default, ZGC/Shenandoah target low pauses.

Q19. What are strong, soft, weak and phantom references?

Strength decides when the GC may clear them: never while strongly reachable; soft under memory pressure; weak at next GC; phantom after finalization for cleanup.

Q20. synchronized vs volatile vs Atomic?

synchronized: mutual exclusion and visibility. volatile: visibility only. Atomic classes: lock-free atomic single-variable updates.

Q21. What causes a deadlock and how do you avoid it?

Circular waiting on locks. Avoid with consistent lock ordering, timeouts (`tryLock`), fewer nested locks.

Q22. Thread vs Runnable vs Callable?

Thread is the execution unit; Runnable is a task without result; Callable returns a result and can throw checked exceptions.

Q23. What are virtual threads?

Cheap JVM-managed threads (Java 21) letting blocking-style code scale to very many concurrent tasks.

Q24. What are lambda expressions and functional interfaces?

A lambda is an inline implementation of an interface with one abstract method (functional interface), e.g. Predicate, Function.

Q25. map vs flatMap in streams?

`map` transforms each element one-to-one; `flatMap` transforms each element to a stream and flattens the results.

Q26. What is Optional for?

To represent a possibly absent return value explicitly, avoiding null checks; not for fields or parameters.

Q27. What are records and sealed classes?

Records are concise immutable data carriers (Java 16). Sealed types restrict permitted subtypes (Java 17), enabling exhaustive switches.

Q28. What is type erasure?

Generic type arguments exist only at compile time; at runtime they are erased to bounds (usually Object).

Q29. What is the difference between shallow and deep copy?

Shallow copies share referenced objects; deep copies duplicate the whole object graph.

Q30. What is dependency injection?

Supplying an object's dependencies from outside (constructor, setter, or a framework like Spring) instead of creating them internally, improving testability.

Q31. Is Java pass-by-value or pass-by-reference?

Always pass-by-value. Object references are passed by value, so methods can mutate the object but not rebind the caller's variable.

Q32. What is the singleton pattern and how do you make it thread-safe?

One instance per JVM. Use an enum singleton, the holder idiom, or eager initialization.

25. Learning Roadmap and Practice Projects

Stage	Focus	Outcome
1. Foundations (2 to 4 weeks)	Syntax, types, operators, control flow, arrays, strings, methods	Solve basic problems (patterns, primes, Fibonacci, string manipulation)
2. OOP (3 to 4 weeks)	Classes, inheritance, polymorphism, interfaces, exceptions, packages	Model a small system in objects (library, bank, parking lot)
3. Core APIs (3 to 4 weeks)	Collections, generics, streams, lambdas, I/O, date-time	Data-processing programs, file-based apps
4. Advanced (4 to 6 weeks)	Concurrency, JVM/GC basics, JDBC, testing, build tools, Git	Multi-threaded utilities, DB-backed console app with tests
5. Frameworks (6+ weeks)	Spring Boot, JPA/Hibernate, REST, security, Docker	Deployable REST backend with a database
6. Depth	System design, microservices, messaging, performance tuning, cloud	Production-grade services; interview readiness

25.1 Project Ideas by Level

- **Beginner:** number-guessing game, calculator, student marks manager, ATM simulation, text-based tic-tac-toe.
- **Intermediate:** library management system (OOP + collections), expense tracker with file persistence, inventory app with JDBC, chat server with sockets.
- **Advanced:** REST API with Spring Boot + PostgreSQL + JWT security, URL shortener, task scheduler with thread pools, in-memory cache with LRU eviction, simple key-value store.
- **Portfolio:** deploy a containerized service (Docker), add CI (GitHub Actions), tests, and API documentation (OpenAPI).

25.2 Practice and Resources

- Practice daily on platforms such as LeetCode, HackerRank, or Codeforces, and implement classic data structures (linked list, stack, BST, heap, graph) yourself in Java.
- Read the official **Java Tutorials** and **JDK API Javadoc** (docs.oracle.com, openjdk.org). Books: *Effective Java* (Joshua Bloch), *Java Concurrency in Practice* (Goetz), *Head First Java*, *Modern Java in Action*.
- Use a good IDE (IntelliJ IDEA, Eclipse, or VS Code with Java extensions), learn the debugger, and use Git from day one.
- Read other people's code (open-source libraries) and build things end to end.

Condensed Summary

The whole guide reduced to the points worth remembering.

Topic	Key points
Platform	Source -> bytecode -> JVM (interpreter + JIT). JDK = JRE + tools. LTS releases: 8, 11, 17, 21, 25. New release every 6 months.
Basics	8 primitives, everything else is a reference. <code>int</code> division truncates; floating point is inexact. Strings are immutable; <code>==</code> vs <code>equals</code> . <code>var</code> infers local types.
Control flow	<code>if</code> , arrow-form <code>switch</code> expressions (Java 14+), pattern-matching <code>switch</code> (21), loops, labeled <code>break/continue</code> .
Arrays & Strings	Fixed-size arrays; <code>Arrays</code> utilities. String pool; <code>StringBuilder</code> in loops; text blocks.
Methods	Pass-by-value always. Overload resolution: exact, widening, boxing, varargs. Recursion needs a base case.
OOP	Encapsulation, inheritance (single), polymorphism (overload = compile time, override = runtime), abstraction (abstract class vs interface). Prefer composition.
Modifiers	<code>private</code> < <code>default</code> < <code>protected</code> < <code>public</code> . <code>static</code> = class-level; <code>final</code> = no reassign/override/extend; <code>abstract</code> ; <code>sealed</code> .
Special types	Nested classes, enums (with fields and methods), records (immutable data), sealed hierarchies, <code>Object</code> methods (<code>equals/hashCode/toString</code>).
Design	SOLID; patterns: Singleton (enum), Factory, Builder, Strategy, Observer; DRY, KISS, YAGNI.
Exceptions	Checked vs unchecked; <code>try/catch/finally</code> ; try-with-resources; custom exceptions; keep the cause; never swallow.
Generics	Compile-time type safety; erasure at runtime; bounded types; wildcards with PECS; no primitives as arguments.
Collections	List / Set / Map / Queue. <code>ArrayList</code> , <code>HashMap</code> , <code>TreeMap</code> , <code>ArrayDeque</code> , <code>PriorityQueue</code> . <code>HashMap</code> : buckets, load factor 0.75, trees at 8. <code>Comparable</code> vs <code>Comparator</code> .
Functional	Lambdas, functional interfaces, method references. Streams: lazy pipeline, <code>filter/map/collect</code> ; <code>Optional</code> for absent values.
APIs	<code>java.time</code> for dates; <code>BigDecimal</code> for money; <code>Files/Path</code> for I/O; serialization is risky.
Concurrency	Executors over raw threads; <code>synchronized</code> , <code>volatile</code> , atomics, locks; <code>ConcurrentHashMap</code> ; <code>CompletableFuture</code> ; virtual threads (21); avoid deadlock.
JVM & memory	Heap (Young/Old), stack, metaspace. G1 default; ZGC for low pause. Tools: <code>jcmd</code> , <code>jstack</code> , JFR, heap dumps. Leaks come from lingering references.
Data & web	JDBC with <code>PreparedStatement</code> and pooling; <code>HttpClient</code> ; Jackson; JUnit 5 + Mockito; SLF4J logging.
Ecosystem	Maven/Gradle, Spring Boot, Hibernate/JPA, Kafka, Docker/Kubernetes, IntelliJ/Eclipse.
Habits	Immutability, interfaces, fail fast, close resources, test everything, measure before optimizing, keep JDK patched.

Quick Cheat Sheet

Need	Use
Compare strings	<code>a.equals(b)</code> or <code>Objects.equals(a, b)</code>
Build strings in a loop	<code>StringBuilder</code>
Resizable list / key lookup / unique items	<code>ArrayList</code> / <code>HashMap</code> / <code>HashSet</code>
Sorted data / stack or queue / priority	<code>TreeMap</code> / <code>TreeSet</code> / <code>ArrayDeque</code> / <code>PriorityQueue</code>
Immutable collection	<code>List.of(...)</code> , <code>Set.of(...)</code> , <code>Map.of(...)</code>
Exact money maths	<code>new BigDecimal("10.25")</code>
Read a file quickly	<code>Files.readString(Path.of("f.txt"))</code>
Run background work	<code>ExecutorService</code> or <code>CompletableFuture</code> ; virtual threads for blocking I/O
Thread-safe counter / map	<code>AtomicInteger</code> or <code>LongAdder</code> / <code>ConcurrentHashMap</code>
Absent value	<code>Optional<T></code> return type
Data holder	<code>record Name(Type field, ...)</code>
Cleanup resources	<code>try (var r = open()) { ... }</code>